

SOA Tech Magazine

Thoughts about Service Oriented Architectures, APIs, Artificial Intelligence, and other interesting trends and topics

A brief look at the evolution of interface protocols leading to modern APIs



February 27, 2019

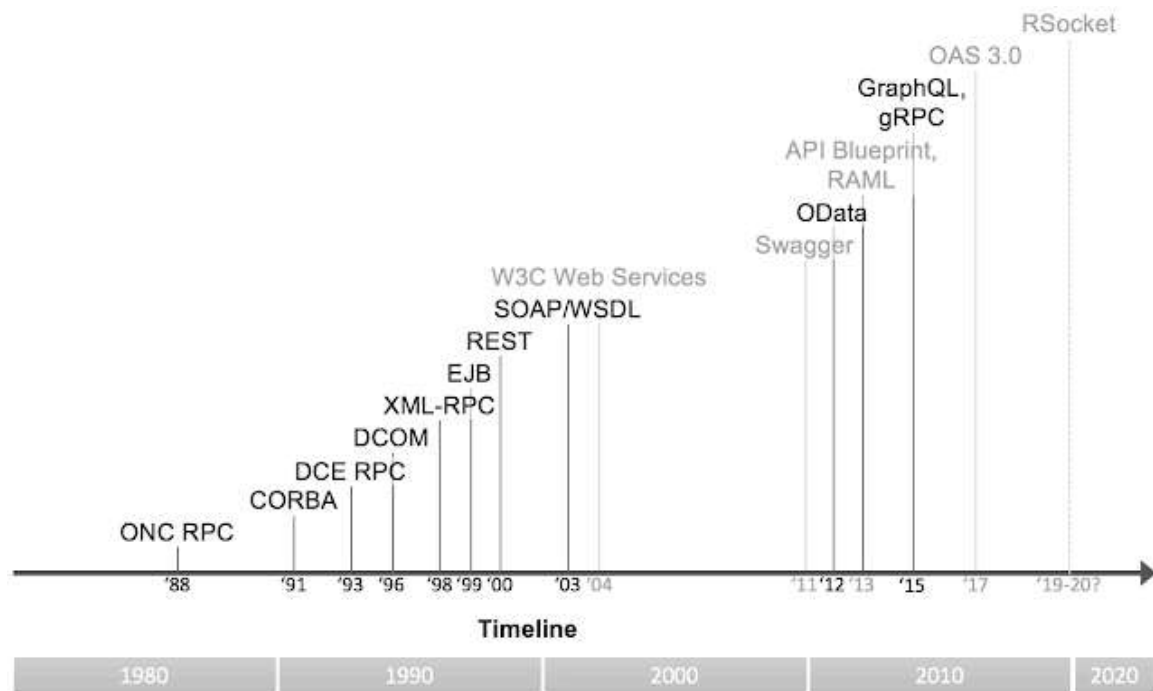
Application interfaces are as old as the origins of distributed computing and can be traced back to the late 1960's when the first request-response style protocols were conceived. For example, according to this research, it wasn't until the late 1980's when the first popular release of RPC (described below) was introduced by SUN Microsystems (later acquired by Oracle), that internet-based interface protocols gained wide popularity and adoption.

This is perhaps why the term **Application Programming Interface (API)** even today can often result in ambiguity depending on who you ask and in what context. This is probably because of the fact that historically the term API has been used to (and to a degree continues to) describe all sorts of interfaces well beyond just web APIs (e.g. REST).

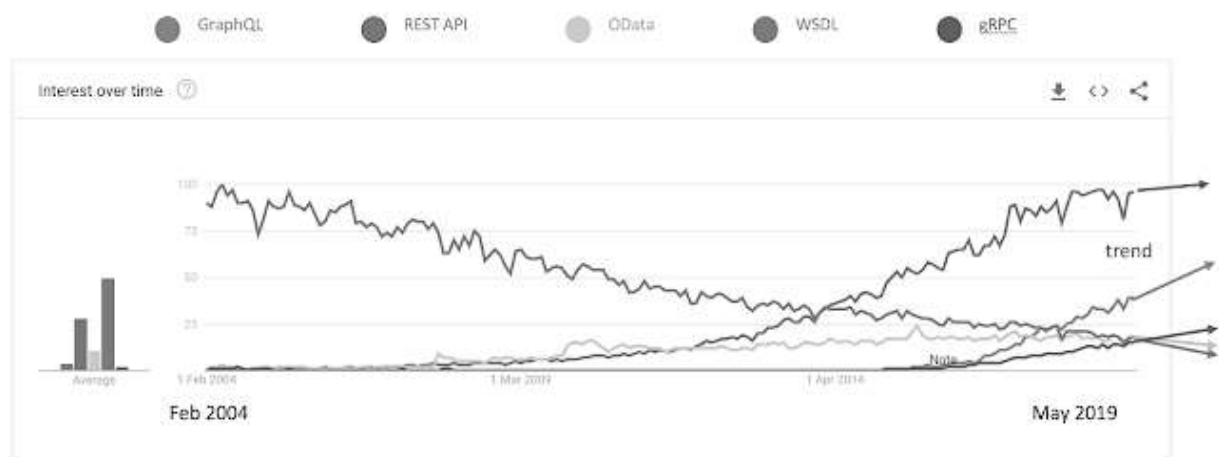
This article therefore attempts to demystify (to an extent) the origins of modern web-based APIs. This is done by listing and describing in chronological order (as illustrated below) the different interface protocols and standards that in my view have had major influence to modern web-based APIs as we know them today (e.g. SOAP/WSDL based web services, REST, GraphQL, gRPC to name a few).

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI



This article is part the research done for my coming book Enterprise API Management where I deep-dive into the 3 most trendy API architectural styles according to the followig Google trends.



Note that some of the texts in the following section are not mine but extracts from the referenced articles. Please do let me know if I missed any reference! Thanks.

ONC RPC

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

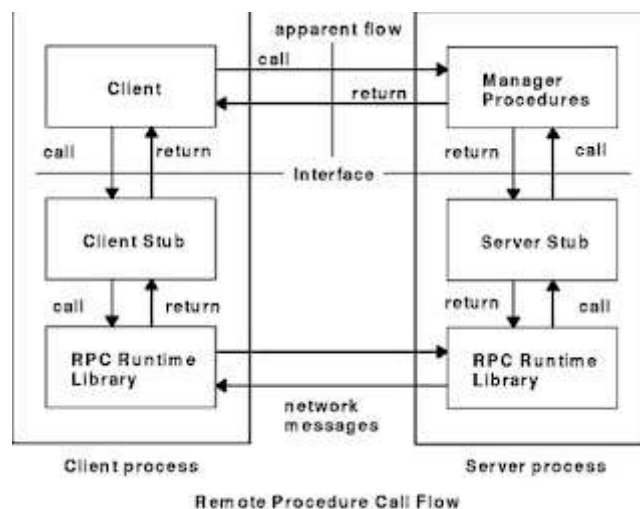
were a normal (local) procedure call, without the programmer explicitly coding the details for the remote interaction. That is, the programmer writes essentially the same code whether the subroutine is local to the executing program, or remote. This is a form of client–server interaction (caller is client, executor is server), typically implemented via a request–response message-passing system.

In the object-oriented programming paradigm, RPC calls are represented by remote method invocation (RMI).

The RPC model implies a level of location transparency, namely that calling procedures is largely the same whether it is local or remote, but usually they are not identical, so local calls can be distinguished from remote calls. Remote calls are usually orders of magnitude slower and less reliable than local calls, so distinguishing them is important.

Note that theoretical proposals of remote procedure calls as the model of network operations date to the 1970s, and practical implementations date to the early 1980s. Bruce Jay Nelson is generally credited with coining the term "remote procedure call" in 1981. Though the idea of treating network operations as remote procedure calls goes back at least to the 1970s in early ARPANET documents. The first popular implementation of RPC on Unix was Sun's RPC.

Architecture



From: [https://cio-wiki.org/wiki/Remote_Procedure_Call_\(RPC\)](https://cio-wiki.org/wiki/Remote_Procedure_Call_(RPC))

Interface definition

External Data Representation (XDR) / RPC language.

Payload

Serialised data based on XDR.

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

First released

April 1988.

References

<https://tools.ietf.org/html/rfc1057>

https://en.wikipedia.org/wiki/Open_Network_Computing_Remote_Procedure_Call

https://en.wikipedia.org/wiki/Remote_procedure_call

<https://searchmicroservices.techtarget.com/definition/Remote-Procedure-Call-RPC>

<https://tools.ietf.org/html/rfc1050>

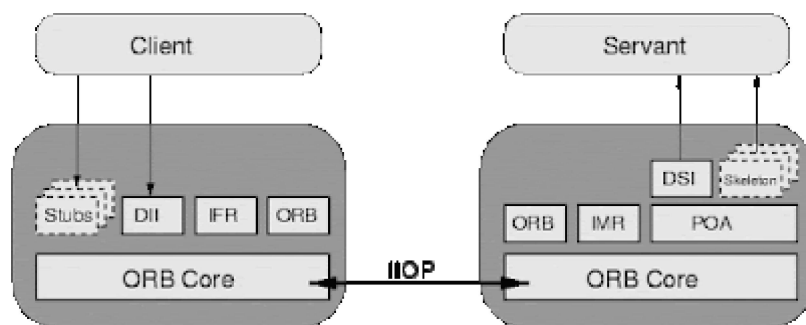
<https://stackoverflow.com/questions/670630/what-is-idl>

https://www.gnu.org/software/guile-rpc/manual/html_node/Quick-Start.html

CORBA

The **Common Object Request Broker Architecture (CORBA)** is a standard defined by the Object Management Group (OMG) and was designed to facilitate the communication of systems that are deployed on diverse platforms.

CORBA enables collaboration between systems on different operating systems, programming languages, and computing hardware. CORBA uses an object-oriented model although the systems that use the CORBA do not have to be object-oriented. CORBA is an example of the distributed object paradigm.

Architecture

From: <https://ados.web.psi.ch/slsnotes/tmeta030225/node2.html>

Interface definition

CORBA uses an interface definition language (IDL) to specify the interfaces that objects present to the outer world. CORBA then specifies a mapping from IDL to a specific implementation language

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

TCP/IP and later HTTP (since 2007 apparently with HTIOP).

First released

Version 1.0 was released in October 1991.

References

https://en.wikipedia.org/wiki/Common_Object_Request_Broker_Architecture

https://en.wikipedia.org/wiki/General_Inter-ORB_Protocol

<http://download.dre.vanderbilt.edu/>

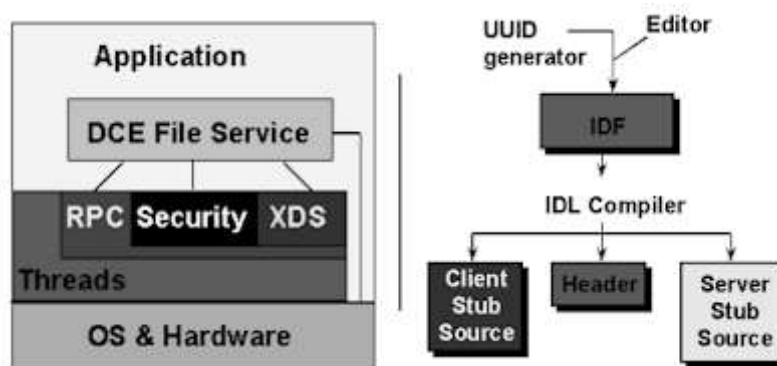
<https://github.com/cflowe/ACE/tree/master/TAO/orbsvcs/orbsvcs/HTIOP>

DCE RPC

The **Distributed Computing Environment (DCE) RPC** was an RPC system commissioned by Open Software Foundation (OSF), a non-profit organisation originally consisting Apollo Computer, Groupe Bull, Digital Equipment Corporation, Hewlett-Packard, IBM, Nixdorf Computer, and Siemens AG, sometimes referred to as the "Gang of Seven". In February 1996 Open Software Foundation merged with X/Open to become The Open Group. The OSF was intended to be a joint development effort mostly in response to a perceived threat of "merged UNIX system" efforts by AT&T Corporation and Sun Microsystems.

In DCE RPC, the client and server stubs are created by compiling a description of the remote interface (interface definition file) with the DCE Interface Definition Language (IDL) compiler. The client application, the client stub, and one instance of the RPC runtime library all execute in the caller machine; the server application, the server stub, and another instance of the RPC runtime library execute in the called (server) machine.

Architecture



Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

Language (IDL).

Payload

The DCE RPC protocol specifies that inputs and outputs be passed in octet streams. Whereas the IDL is to provide syntax for describing these structured data types and values. The Network Data Representation (NDR) specification of the protocol is responsible to provide a mapping of IDL data types onto octet streams. NDR defines primitive data types, constructed data types and representations for these types in an octet stream.

Transport protocol

DCE/RPC can run atop a number of protocols, including:

- TCP: Typically, connection oriented DCE/RPC uses TCP as its transport protocol. The well known TCP port for DCE/RPC EPMAP is 135. This transport is called ncacn_ip_tcp.
- UDP: Typically, connectionless DCE/RPC uses UDP as its transport protocol. The well known UDP port for DCE/RPC EPMAP is 135. This transport is called ncadg_ip_udp.
- SMB: Connection oriented DCE/RPC can also use authenticated named pipes on top of SMB as its transport protocol. This transport is called ncacn_np.
- SMB2: Connection oriented DCE/RPC can also use authenticated named pipes on top of SMB2 as its transport protocol. This transport is called ncacn_np.

First released

The first release ("P312 DCE: Remote Procedure Call") dates to 1993.

References

<https://en.wikipedia.org/wiki/DCE/RPC>

<https://wiki.wireshark.org/DCE/RPC>

<https://www.dcerpc.org/documentation/rpc-porting.pdf>

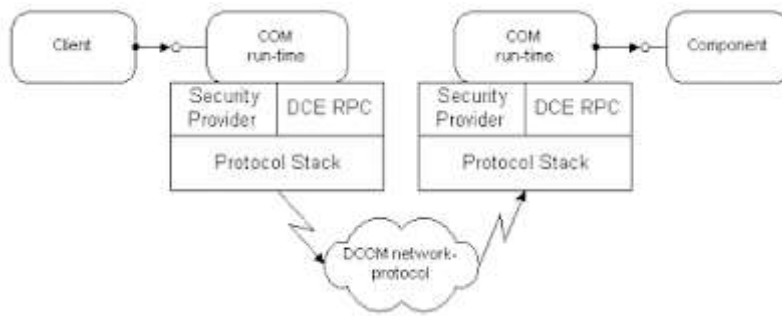
<http://pubs.opengroup.org/onlinepubs/9629399/toc.pdf>

DCOM

The **Distributed Component Object Model (DCOM)** is a proprietary Microsoft technology for communication between software components on networked computers. The addition of the "D" to COM was due to extensive use of DCE/RPC (Distributed Computing Environment/Remote Procedure Calls) – more specifically Microsoft's enhanced version,

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI



From: <http://active-undelete.com/dcom-overview.htm>

Interface definition

Characteristics of an interface are defined in an interface definition (IDL) file and an optional application configuration file (ACF):

- The IDL file specifies the characteristics of the application's interfaces on the wire — that is, how - data is to be transmitted between client and server, or between COM objects.
- The ACF file specifies interface characteristics, such as binding handles, that pertain only to the local operating environment. The ACF file can also specify how to marshal and transmit a complex data structure in a machine-independent form.
- The IDL and ACF files are scripts written in Microsoft Interface Definition Language (MIDL), which is the Microsoft implementation and extension of the OSF-DCE interface definition language (IDL).

Payload

DCOM objects (Microsoft proprietary).

Transport protocol

TCP/IP and later HTTP (since 2003).

First released

OLE 1.0, released in 1990, was an evolution of the original Dynamic Data Exchange (DDE) concept that Microsoft developed for earlier versions of Windows. OLE 1.0 later evolved to become an architecture for software components known as the Component Object Model (COM), which later in early 1996 became DCOM.

References

https://en.wikipedia.org/wiki/Distributed_Component_Object_Model

<https://www.varonis.com/blog/dcom-distributed-component-object-model/>

https://en.wikipedia.org/wiki/Object_Linking_and_Embedding#History

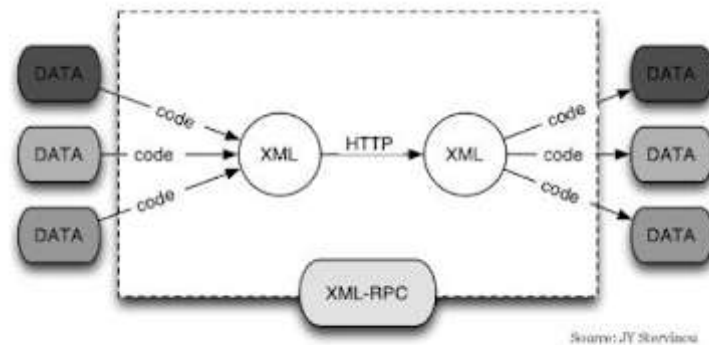
<https://docs.microsoft.com/en-us/windows/desktop/midl/com-dcom-and-typing-libraries>

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

The **Extensible Markup Language (XML) Remote Procedure Call (RPC)** is a protocol which uses XML to encode its calls and HTTP as a transport mechanism. In XML-RPC, a client performs an RPC by sending an HTTP request to a server that implements XML-RPC and receives the HTTP response. A call can have multiple parameters and one result. The protocol defines a few data types for the parameters and result. Some of these data types are complex, i.e. nested. For example, you can have a parameter that is an array of five integers.

Architecture



From: <http://xmlrpc.scripting.com/>

Interface definition

No explicit interface definition language. The protocol defines a set of header and payload requirements which the implementation (e.g. in Java) must comply with.

Payload

XML.

Transport protocol

HTTP.

First released

The XML-RPC protocol was created in 1998 by Dave Winer of UserLand Software and Microsoft, with Microsoft seeing the protocol as an essential part of scaling up its efforts in business-to-business e-commerce. As new functionality was introduced, the standard evolved into what is now SOAP.

References

<https://en.wikipedia.org/wiki/XML-RPC>

<http://xmlrpc.scripting.com/>

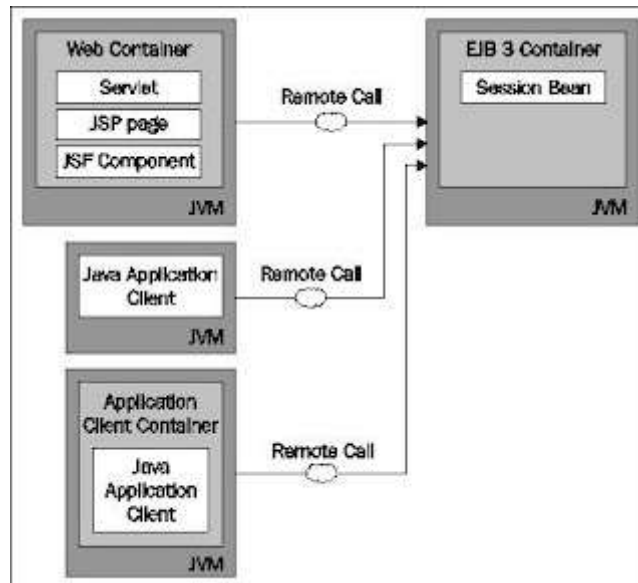
Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

EJB

Enterprise Java Beans (EJB) is a server-side software component that encapsulates business logic of an application. An EJB web container provides a runtime environment for web related software components, including computer security, Java servlet lifecycle management, transaction processing, and other web services. The EJB specification is a subset of the Java EE specification. EJBs are/were typically used when building highly scalable and robust enterprise level applications that can be deployed on Jakarta EE (former J2EE) compliant Application Server such as JBOSS, Web Logic etc.

Architecture



From: https://subscription.packtpub.com/book/application_development/9781847195609/1/ch01lv11sec02/the-ejb-3-architecture

Interface definition

Java remote interface (extending `javax.ejb.EJBObject`) declaring the methods that a client can invoke.

Payload

Originally as serialised Java objects (e.g. DTO), but later releases also support XML and JSON over HTTP.

Transport protocol

EJB originally specified Java Remote Method Invocation (RMI) as the transport protocol, but later releases also support HTTP.

First release

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

https://en.wikipedia.org/wiki/Enterprise_JavaBeans

https://docs.oracle.com/cd/A97335_02/apps.102/a83725/ejb2.htm

<https://www.oreilly.com/library/view/enterprise-javabeans-31/9781449399139/ch04.html>

<https://www.tutorialspoint.com/ejb/index.htm>

<https://docs.oracle.com/javaee/7/api/javax/ejb/EJBObject.html>

https://docs.oracle.com/cd/A97335_02/apps.102/a83725/overvie1.htm

https://en.wikipedia.org/wiki/Java_remote_method_invocation

http://www.adam-bien.com/roller/abien/entry/javaee_7_retired_the_dto

REST

Representational State Transfer (REST) is a software architectural style that defines a set of constraints to be used for creating Web services. Such constraints restrict the ways that the server can process and respond to client requests so that, by operating within these constraints, the system gains desirable non-functional properties, such as performance, scalability, simplicity, modifiability, visibility, portability, and reliability. If a system violates any of the required constraints, it cannot be considered RESTful.

These constraints are (from Roy's dissertation):

- **Client-server:** separation of concerns is the principle behind the client-server constraints. By separating the user interface concerns from the data storage concerns, we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components. Perhaps most significant to the Web, however, is that the separation allows the components to evolve independently, thus supporting the Internet-scale requirement of multiple organisational domains
- **Stateless:** communication must be stateless in nature such that each request from client to server must contain all of the information necessary to understand the request, and cannot take advantage of any stored context on the server. Session state is therefore kept entirely on the client.
- **Cache:** In order to improve network efficiency, we add cache constraints to form the client-cache-stateless-server style. Cache constraints require that the data within a response to a request be implicitly or explicitly labeled as cacheable or non-cacheable. If a response is cacheable, then a client cache is given the right to reuse that response data for later, equivalent requests
- **Uniform interface:** The central feature that distinguishes the REST architectural style from other network-based styles is its emphasis on a uniform interface between components. By

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

- **Layered system:** In order to further improve behaviour for Internet-scale requirements the layered system style allows an architecture to be composed of hierarchical layers by constraining component behaviour such that each component cannot "see" beyond the immediate layer with which they are interacting. By restricting knowledge of the system to a single layer, we place a bound on the overall system complexity and promote substrate independence. Layers can be used to encapsulate legacy services and to protect new services from legacy clients, simplifying components by moving infrequently used functionality to a shared intermediary. Intermediaries can also be used to improve system scalability by enabling load balancing of services across multiple networks and processors.
- **Code-on-demand:** REST allows client functionality to be extended by downloading and executing code in the form of applets or scripts. This simplifies clients by reducing the number of features required to be pre-implemented. Allowing features to be downloaded after deployment improves system extensibility. However, it also reduces visibility, and thus is only an optional constraint within REST.

REST was first introduced in the year 2000 as part of Roy Fielding's PhD dissertation titled "Architectural Styles and the Design of Network-based Software Architectures".

REST was first introduced in the year 2000 as part of Roy Fielding's PhD dissertation titled "Architectural Styles and the Design of Network-based Software Architectures". Although the first (or at least the first publicly known) REST API was launched by eBay the same year, the adoption of REST over alternatives (such as SOAP/WSDL Web Services) really only gained traction towards the end of 2004 when Flickr first launched its first publicly available REST API and shortly after Facebook and Twitter followed also publishing their own public REST APIs.

Note that the first (or at least the first publicly known) REST API launched by eBay the same year as Roy's dissertation.

Architecture

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

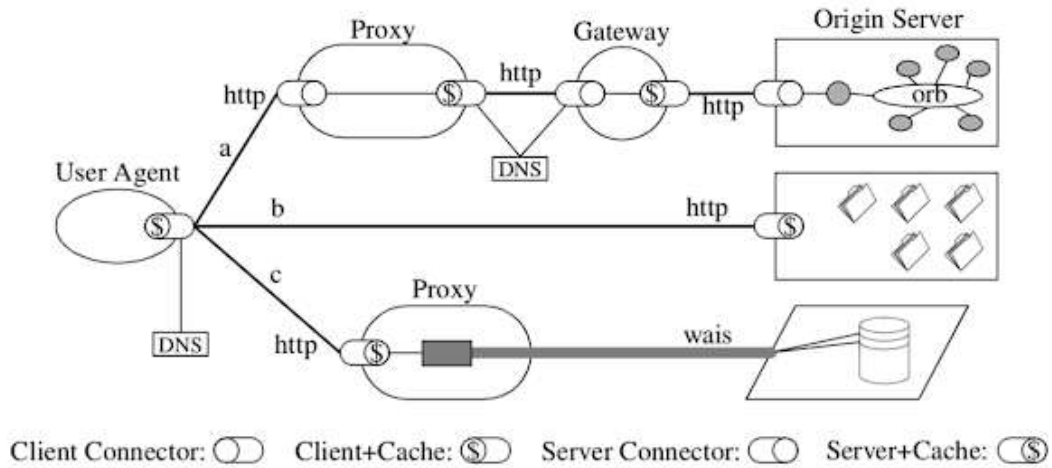


Figure 5-10. Process View of a REST-based Architecture

A user agent is portrayed in the midst of three parallel interactions: a, b, and c. The interactions were not satisfied by the user agent's client connector cache, so each request has been routed to the resource origin according to the properties of each resource identifier and the configuration of the client connector. Request (a) has been sent to a local proxy, which in turn accesses a caching gateway found by DNS lookup, which forwards the request on to be satisfied by an origin server whose internal resources are defined by an encapsulated object request broker architecture. Request (b) is sent directly to an origin server, which is able to satisfy the request from its own cache. Request (c) is sent to a proxy that is capable of directly accessing WAIS, an information service that is separate from the Web architecture, and translating the WAIS response into a format recognized by the generic connector interface. Each component is only aware of the interaction with their own client or server connectors; the overall process topology is an artifact of our view.

From: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Interface definition

Many open-sourced interface definition languages exists for REST APIs -although none as part of the original Roy Fielding's REST PhD dissertation.

Some of the most popular ones are REST are:

- Swagger -later renamed to OpenAPI Specification (OAS) being the latest version 3.0 (with Swagger yous becoming the name to refer to a set of toolsets for adopting OAS).
- API blueprint created by the founders of Apiary.io (and now part of Oracle) which is a platform that offers robust REST API design and testing capabilities.
- RESTful API Modeling Language (RAML) created by MuleSoft.

Payload:

REST does not specify any specific payload format, however majority of REST APIs make use of the JavaScript Object Notation (JSON) which is a lightweight data-interchange format easy for humans to read and write. XML payloads are also not uncommon in REST

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

First release:

REST was defined by Roy Fielding in his 2000 PhD dissertation "Architectural Styles and the Design of Network-based Software Architectures" at UC Irvine. He developed the REST architectural style in parallel with HTTP 1.1 of 1996–1999, based on the existing design of HTTP 1.0 of 1996.

In terms of interface definition languages:

- Swagger was first released in 2011. Its name switched to OAS in 2016..
- In 2017, OAS 3.0 was released.
- API Blueprint and RAML were both released in 2013.

References:

<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

https://en.wikipedia.org/wiki/Representational_state_transfer

<https://blog.readme.io/the-history-of-rest-apis/>

<https://thehistoryoftheweb.com/ebay-apis-connected-web/>

[https://en.wikipedia.org/wiki/Overview_of_RESTful_API_Description_Languages#List_of_R
ESTful_API_DLs](https://en.wikipedia.org/wiki/Overview_of_RESTful_API_Description_Languages#List_of_RESTful_API_DLs)

https://en.wikipedia.org/wiki/OpenAPI_Specification#History

SOAP/WSDL & Web Services

The **Simple Object Access Protocol (SOAP)** is an XML based protocol for exchange of information in a decentralised, distributed environment. SOAP was designed as an object-access protocol in 1998 for Microsoft.

It consists of three parts:

- An envelope that defines a framework for describing what is in a message and how to process it
- A set of encoding rules for expressing instances of application-defined datatypes
- A convention for representing remote procedure calls and responses.

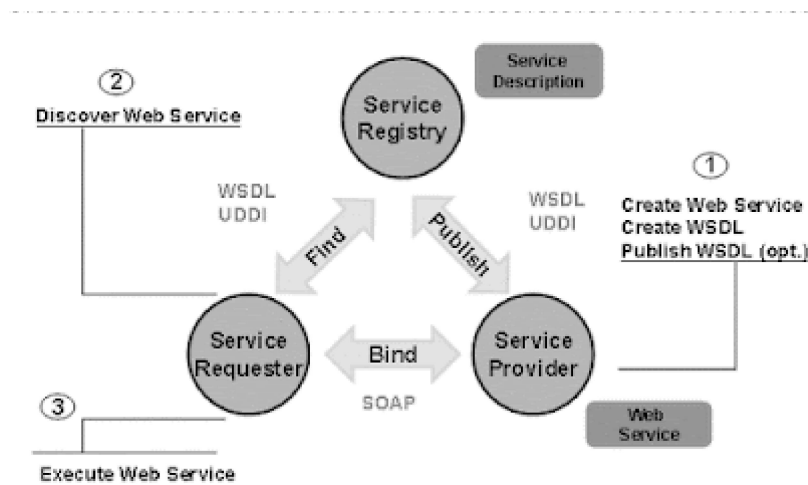
The **Web Services Description Language (WSDL)** as its name suggests is an interface description language also based on XML. The main purpose of WSDL is to describe the

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

Its worth mentioning that Web Services became one of the core building blocks of **Service Oriented Architectures (SOA)**.

Architecture



From: <https://thecustomizewindows.com/2013/08/web-services-what-are-web-services/>

Interface definition

WSDL.

Payload:

XML SOAP message containing an envelop and within the header and body.

Transport protocol:

SOAP can potentially be used in combination with a variety of other protocols; however, the only bindings defined in this document describe how to use SOAP in combination with HTTP and HTTP Extension Framework.

First release:

Both SOAP and WSDL versions 1.2 became an official W3C recommendation in June 2003. In 2004 the term Web Services became official as part of W3C's Web Service Architecture recommendation.

References:

<https://www.w3.org/TR/2000/NOTE-SOAP-20000508/>

<https://www.w3.org/TR/ws-arch/>

<https://en.wikipedia.org/wiki/SOAP>

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

The **Open Data Protocol (OData)** is a REST-based protocol originally designed by Microsoft but later becoming ISO/IEC approved and an OASIS standard. The main objective of OData is to standardise the way in which basic data access operations can be made available via REST.

OData It's built on top of HTTP and uses URLs to address and access data feed resource (in JSON format). The protocol is based on AtomPub and extends it by adding metadata to describe the data source and a standard means of querying the underlying data.

OData was subject to criticism in 2013 when Netflix abandoned the use of the protocol.

"A more technical concern with OData is that it encourages poor development and API practices by providing a black-box framework to enforce a generic repository pattern. This can be regarded as an anti-pattern as it provides both a weak contract and leaky abstraction. An API should be designed with a specific intent in mind rather than providing a generic set of methods that are automatically generated. OData tends to give rise to very noisy method outputs with a metadata approach that feels more like a WSDL than REST. This doesn't exactly foster simplicity and usability".

In spite of this, large software companies like Microsoft and SAP still back the protocol, although industry wide OData seems to have declined in popularity.

Architecture



From: <https://www.progress.com/blogs/odata-faqs-why-should-rest-api-developers-use-odata>

Interface definition

OData services are described in terms of an Entity Model. The Common Schema Definition Language (CSDL) defines a representation of the entity model exposed by an OData service using (in version 4) JSON.

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

First release:

In May, 2012, companies including Citrix, IBM, Microsoft, Progress Software, SAP AG, and WSO2 submitted a proposal to OASIS to begin the formal standardization process for OData. Many Microsoft products and services support OData, including Microsoft SharePoint, Microsoft SQL Server Reporting Services, and Microsoft Dynamics CRM. OData V4.0 was officially approved as a new OASIS standard in March, 2014.

References:

<https://www.webopedia.com/TERM/O/odata-open-data-protocol.html>

<https://www.oasis-open.org/committees/download.php/46906/odata-v1.0-wd01-2012-9-12.doc>

https://en.wikipedia.org/wiki/Open_Data_Protocol

<https://www.odata.org/>

http://docs.oasis-open.org/odata/odata-csdl-json/v4.01/cs01/odata-csdl-json-v4.01-cs01.html#_Toc505948162

<https://www.ben-morris.com/netflix-has-abandoned-odata-does-the-standard-have-a-future-without-an-ecosystem/>

GraphQL

The **Graph Query Language (GraphQL)** is a query language for APIs and a runtime for fulfilling those queries with your existing data. It was created by Facebook in 2012 to get around a common constraints in the REST approach when fetching data.

GraphQL provides a complete and understandable description of the data in your API, gives clients the power to ask for exactly what they need and nothing more, makes it easier to evolve APIs over time, and enables powerful developer tools.

GraphQL isn't tied to any specific database or storage engine and is instead backed by your existing code and data. A GraphQL service is created by defining types and fields on those types, then providing functions for each field on each type.

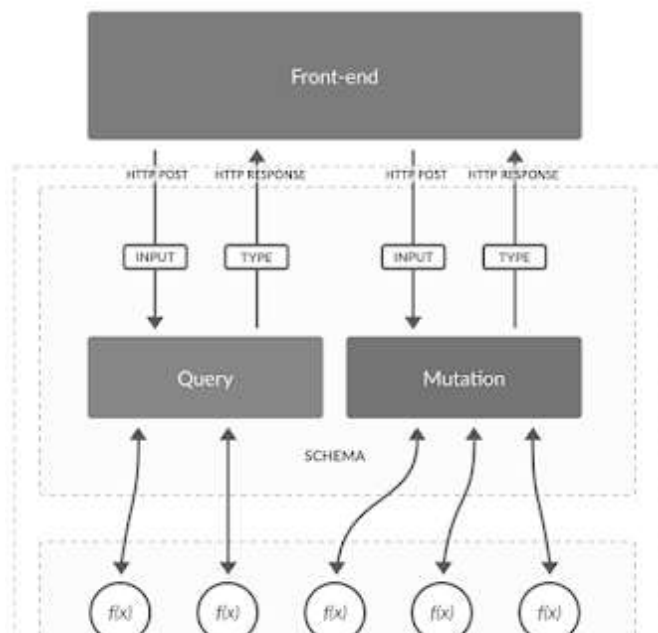
GraphQL is not a programming language capable of arbitrary computation, but is instead a language used to query application servers that have capabilities defined in this specification. GraphQL does not mandate a particular programming language or storage system for application servers that implement it. Instead, application servers take their

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

- **Hierarchical:** Most product development today involves the creation and manipulation of view hierarchies. To achieve congruence with the structure of these applications, a GraphQL query itself is structured hierarchically. The query is shaped just like the data it returns. It is a natural way for clients to describe data requirements.
- **Product-centric:** GraphQL is unapologetically driven by the requirements of views and the front-end engineers that write them. GraphQL starts with their way of thinking and requirements and build the language and runtime necessary to enable that.
- **Strong-typing:** Every GraphQL server defines an application-specific type system. Queries are executed within the context of that type system. Given a query, tools can ensure that the query is both syntactically correct and valid within the GraphQL type system before execution, i.e. at development time, and the server can make certain guarantees about the shape and nature of the response.
- **Client-specified queries:** Through its type system, a GraphQL server publishes the capabilities that its clients are allowed to consume. It is the client that is responsible for specifying exactly how it will consume those published capabilities. These queries are specified at field-level granularity. In the majority of client-server applications written without GraphQL, the server determines the data returned in its various scripted endpoints. A GraphQL query, on the other hand, returns exactly what a client asks for and no more.
- **Introspective:** GraphQL is introspective. A GraphQL server's type system must be queryable by the GraphQL language itself, as will be described in this specification. GraphQL introspection serves as a powerful platform for building common tools and client software libraries.

Architecture



Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

Interface definition

The GraphQL Schema Definition Language (SDL).

Payload:

JSON.

Transport protocol:

HTTP.

First release:

GraphQL was publicly released in 2015. The GraphQL Schema Definition Language (SDL) added to spec in Feb'18.

References:

<https://graphql.org/>

<https://en.wikipedia.org/wiki/GraphQL>

<https://facebook.github.io/graphql/>

<https://www.slideshare.net/luisw19/devoxx-uk-2018-graphql-as-an-alternative-approach-to-rest-96694252>

<https://www.youtube.com/watch?v=hJOOdCPIXbU>

gRPC

gRPC is defined by Google as a modern Remote Procedure Call (RPC) framework that can run in any environment. gRPC in principle enables client and server applications to communicate transparently, and makes it easier to build connected systems.

gRPC was designed based on the following principles:

- **Services not objects, messages not references:** promote the microservices design philosophy of coarse-grained message exchange between systems while avoiding the pitfalls of distributed objects and the fallacies of ignoring the network.
- **Coverage & simplicity:** the stack should be available on every popular development platform and easy for someone to build for their platform of choice. It should be viable on CPU & memory limited devices.
- **Free & open:** make the fundamental features free for all to use. Release all artefacts as open-source efforts with licensing that should facilitate and not impede adoption.

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

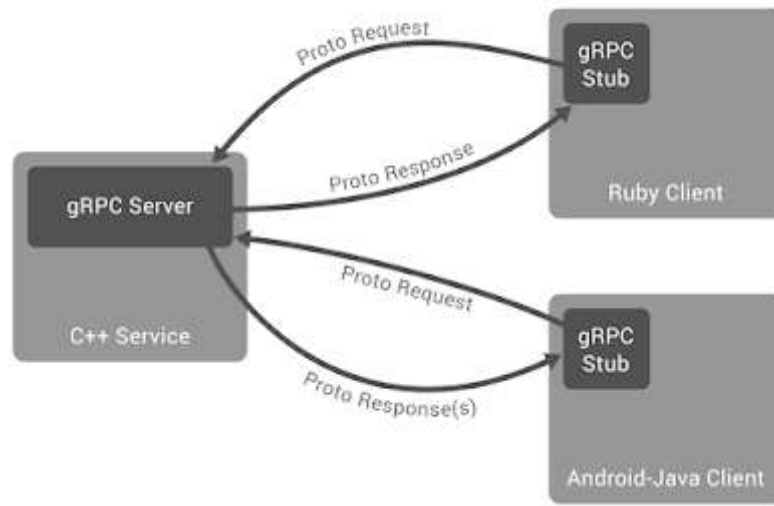
SAIBA MAIS ENTENDI

- **Payload agnostic:** different services need to use different message types and encodings such as protocol buffers, JSON, XML, and Thrift; the protocol and implementations must allow for this. Similarly the need for payload compression varies by use-case and payload type: the protocol should allow for pluggable compression mechanisms.
- **Streaming:** storage systems rely on streaming and flow-control to express large data-sets. Other services, like voice-to-text or stock-tickers, rely on streaming to represent temporally related message sequences.
- **Blocking & non-blocking:** support both asynchronous and synchronous processing of the sequence of messages exchanged by a client and server. This is critical for scaling and handling streams on certain platforms.
- **Cancellation & timeout:** operations can be expensive and long-lived - cancellation allows servers to reclaim resources when clients are well-behaved. When a causal-chain of work is tracked, cancellation can cascade. A client may indicate a timeout for a call, which allows services to tune their behaviour to the needs of the client.
- **Lameducking:** servers must be allowed to gracefully shut-down by rejecting new requests while continuing to process in-flight ones.
- **Flow-control:** computing power and network capacity are often unbalanced between client & server. Flow control allows for better buffer management as well as providing protection from DOS by an overly active peer.
- **Pluggable:** A wire protocol is only part of a functioning API infrastructure. Large distributed systems need security, health-checking, load-balancing and failover, monitoring, tracing, logging, and so on. Implementations should provide extensions points to allow for plugging in these features and, where useful, default implementations.
- **Extensions as APIs:** extensions that require collaboration among services should favour using APIs rather than protocol extensions where possible. Extensions of this type could include health-checking, service introspection, load monitoring, and load-balancing assignment.
- **Metadata exchange:** common cross-cutting concerns like authentication or tracing rely on the exchange of data that is not part of the declared interface of a service. Deployments rely on their ability to evolve these features at a different rate to the individual APIs exposed by services.
- **Standardised status codes:** clients typically respond to errors returned by API calls in a limited number of ways. The status code namespace should be constrained to make these error handling decisions clearer. If richer domain-specific status is needed the metadata exchange mechanism can be used to provide that.

Architecture

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI



From: <https://grpc.io/docs/guides/>

Interface definition

gRPC can use protocol buffers as both its Interface Definition Language (IDL) and as its underlying message interchange format. Protocol buffers is Google's mature open source mechanism for serialising structured data.

Payload:

By default gRPC uses protocol buffers (although it can be used with other data formats such as JSON).

Transport protocol:

HTTP/2.

First release:

Google released gRPC as open source in 2015.

References:

<https://opensource.google.com/projects/grpc>

<https://grpc.io/blog/principles>

<https://grpc.io/faq/>

<https://opensource.com/bus/15/3/google-grpc-open-source-remote-procedure-calls>

<https://developers.google.com/protocol-buffers/docs/overview>

RSocket

RSocket is a reactive, bidirectional, streaming, and reliable communication protocol.

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

RSocket is a bi-directional, multiplexed, message-based, binary protocol based on reactive streams back pressure. It enables the following symmetric interaction models via async message passing over a single connection:

- request/response (stream of 1).
- request/stream (finite stream of many).
- fire-and-forget (no response).
- channel (bi-directional streams).

It also supports session resumption, to allow resuming long-lived streams across different transport connections. This is particularly useful for mobile <> server communication when network connections drop, switch, and reconnect frequently.

Some of the key motivations behind rSocket include:

- support for interaction models beyond request/response such as streaming responses and push.
- application-level flow control semantics (async pull/push of bounded batch sizes) across network boundaries.
- binary, multiplexed use of a single connection.
- support resumption of long-lived subscriptions across transport connections.
- need of an application protocol in order to use transport protocols such as WebSockets and Aeron.

The protocol is specifically designed to work well with Reactive-style applications, which are fundamentally non-blocking and often (but not always) paired with asynchronous behaviour. The use of Reactive back pressure, the idea that a publisher cannot send data to a subscriber until that subscriber has indicated that it is ready, is a key differentiator from "async".

Interface definition

Depends on the implementation. For example RSocket RPC uses Google's protocol buffer v3 as its interface definition language.

Payload:

RSocket provides mechanisms for applications to distinguish payload into two types. Data and Metadata. The distinction between the types in an application is left to the application.

The following are features of Data and Metadata:

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS ENTENDI

Transport protocol:

The RSocket protocol uses a lower level transport protocol to carry RSocket frames. A transport protocol MUST provide the following:

- 1- Unicast reliable delivery.
- 2- Connection-oriented and preservation of frame ordering. Frame A sent before Frame B MUST arrive in source order. i.e. if Frame A is sent by the same source as Frame B, then Frame A will always arrive before Frame B. No assumptions about ordering across sources is assumed.
- 3- FCS is assumed to be in use either at the transport protocol or at each MAC layer hop. But no protection against malicious corruption is assumed.

RSocket as specified here has been designed for and tested with TCP, WebSockets, Aeron and HTTP/2 streams as transport protocols.

First release:

Although originally released by Netflix in October 2015, the protocol is currently a draft for the final specifications. Current version of the protocol is 0.2 (Major Version: 0, Minor Version: 2). This is currently considered a 1.0 Release Candidate. Final testing is being done in Java and C++ implementations with goal to release 1.0 in the near future.

References:

<http://rsocket.io/>

<http://rsocket.io/docs/FAQ>

<https://github.com/rsocket>

<https://en.wikipedia.org/wiki/Rsocket>

<https://www.infoq.com/news/2018/10/rsocket-facebook>

<https://www.infoq.com/articles/give-rest-a-rest-rsocket>

<https://github.com/rsocket/rsocket-rpc-java/blob/master/docs/motivation.md>

<https://github.com/rsocket/rsocket-rpc-java/blob/master/docs/get-started.md>

<https://www.slideshare.net/SpringCentral/pointtopoint-messaging-architecturethe-reactive-endgame>

[API Blueprint](#)[APIs](#)[CORBA](#)[DCOM](#)[EJB](#)[GraphQL](#)[gRPC](#)[OAS](#)[ONC RPC](#)[RAML](#)[REST](#)[RPC](#)[RSocket](#)[SOAP](#)[Swagger](#)

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS **ENTENDI**

To leave a comment, click the button below to sign in with Google.

SIGN IN WITH GOOGLE



 Powered by Blogger



LUIS AUGUSTO WEIR

Leamington Spa, Warwickshire ,
United Kingdom

VISIT PROFILE

Latest Posts

1 **API Trends for 2024: A Glimpse into the Future of Interface Technologies**

In 2023, the surge of Generative AI captured most of the spotlight in the tech ... read more

2 **What is API Product Management?**

Recently I've noticed some good discussions in the API community around what ... read more

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS **ENTENDI**

4 2020 State of the API Report - My Own Thoughts

This morning whilst going through my emails, noticed in my inbox an email ... read more

5 A brief look at the evolution of interface protocols leading to modern APIs

Application interfaces are as old as the origins of distributed computing and ... read more

Archive



Report Abuse

Este site usa os cookies do Google para fornecer serviços e analisar o tráfego. Seu endereço IP e user agent são compartilhados com o Google, além das métricas de segurança e desempenho, para garantir a qualidade de serviço, gerar estatísticas de uso e detectar e eliminar abusos.

SAIBA MAIS **ENTENDI**